

Homework: Learning Audio Representations with Self-Supervision

In this homework, we will consolidate the knowledge gained during the seminar and further explore methods for self-supervised learning of audio representations. During the seminar, we implemented and trained a model based on contrastive learning (InfoNCE). Now, we will extend this work by implementing non-contrastive learning (NCL) approaches and comparing them with previously studied methods.

We will examine how different training paradigms — supervised, contrastive, and non-contrastive — affect the quality of learned embeddings and the stability of the training process

Multi-Format Contrastive Learning of Audio Representations(Paper from seminar)

The core idea of this approach is to learn robust audio embeddings by contrasting different *formats* (or views) of the same audio sample.

For example, one branch may encode the **raw waveform (1D)** while another encodes its **spectrogram (2D)**.

By applying a contrastive loss (InfoNCE), the model is trained to:

- **Pull together** embeddings from different formats of the same audio,
- **Push apart** embeddings from different audio samples.

This multi-format setup encourages the encoder to capture **shared semantic content** across input representations, leading to more general and transferable audio features.

[paper1](#), [paper2](#)

[github source](#)

AudioSet

Dataset with 10 numbers pronounced

```
!git clone https://github.com/soerenab/AudioMNIST.git
```

Task 1 Supervised training (2 points)

Implement straightforward classifier training based on waveforms or spectrogram(use models from seminar). Train it in supervised manner and compute accuracy

```
import random
import numpy as np
```

```

import torch
import os

# Фиксация всех источников случайности
def set_seed(seed=42):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)

    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False

    os.environ['PYTHONHASHSEED'] = str(seed)

set_seed(42)

import torchaudio
import torchaudio.transforms as T
import matplotlib.pyplot as plt
from IPython.display import Audio
from pathlib import Path
from omegaconf import DictConfig

import librosa
import torch.nn as nn
import torch.nn.functional as F
from torch import optim
from tqdm.notebook import tqdm
from torch.utils.data import Subset, DataLoader
from torchaudio import transforms
from sklearn.manifold import TSNE

import warnings
warnings.filterwarnings('ignore')

plt.rcParams.update({'font.size': 14})

# !git clone https://github.com/soerenab/AudioMNIST.git
# !mkdir models

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Available device:", device)

Available device: cuda

root = 'AudioMNIST/data'

# Посмотрим случайные аудиофайлы одного класса от разных спикеров

```

```

def get_random_audio(num_samples=2):
    first_speaker = random.choice(os.listdir(root))
    rand_audio = random.choice(os.listdir(os.path.join(root,
first_speaker)))
    target_class = rand_audio[0]
    print(f'Class #{target_class}')

    for _ in range(num_samples):
        rand_speaker = random.choice(os.listdir(root))

        files = os.listdir(os.path.join(root, rand_speaker))
        rand_audio = random.choice([file for file in files if file[0]
== target_class])

        print(f'Speaker #{rand_speaker}')

        y, sample_rate = librosa.load(
            os.path.join(root, rand_speaker, rand_audio), sr=None,
mono=True)

        waveform = torch.from_numpy(y).float().unsqueeze(0)
        waveform = waveform / (waveform.abs().max() + 1e-8)

        # waveform, sample_rate = torchaudio.load(
        #     os.path.join(root, rand_speaker, rand_audio),
normalize=True
        # )

        print(f'Sample rate: {sample_rate}, waveform shape:
{waveform.shape}')
        display(Audio(waveform, rate=sample_rate))

        # Вернем последнюю выбранную аудиозапись
        return waveform, sample_rate

```

```

waveform, sample_rate = get_random_audio(3)

```

```

Class #1

```

```

Speaker #02

```

```

Sample rate: 48000, waveform shape: torch.Size([1, 26298])

```

```

<IPython.lib.display.Audio object>

```

```

Speaker #18

```

```

Sample rate: 48000, waveform shape: torch.Size([1, 24505])

```

```

<IPython.lib.display.Audio object>

```

```

Speaker #15

```

```

Sample rate: 48000, waveform shape: torch.Size([1, 22012])

```

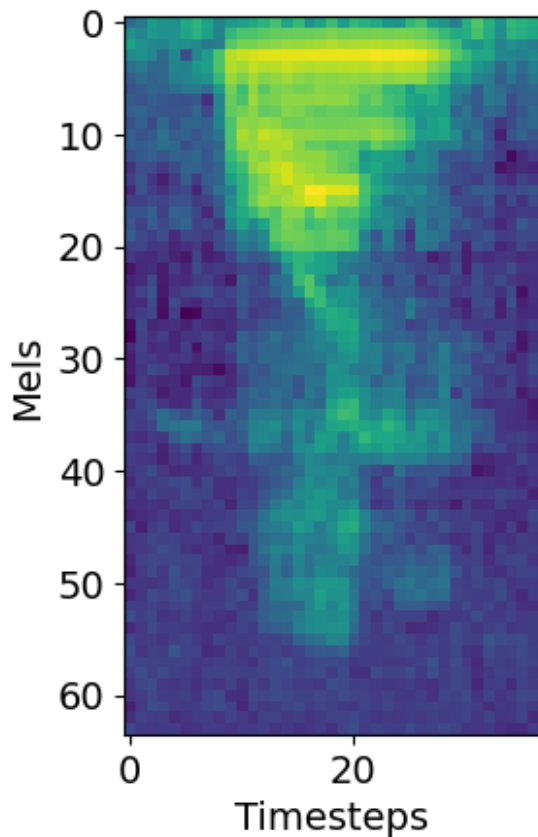
```
<IPython.lib.display.Audio object>
```

```
new_sample_rate = 16000  
resample = T.Resample(orig_freq=sample_rate, new_freq=new_sample_rate)  
resampled_waveform = resample(waveform)
```

```
Audio(resampled_waveform, rate=new_sample_rate)
```

```
<IPython.lib.display.Audio object>
```

```
class LogMelSpectrogram(T.MelSpectrogram):  
    def __init__(self, eps=1e-8, **kwargs):  
        super().__init__(**kwargs)  
        self.eps = eps  
  
    def forward(self, waveform):  
        return (super().forward(waveform) + self.eps).log()  
  
spectrogram = LogMelSpectrogram(sample_rate=16000, n_mels=64)  
logmel_spec = spectrogram(resampled_waveform.squeeze())  
print(f'Spectrogram shape: {logmel_spec.shape}')  
print(f'Values range: ({logmel_spec.min()}, logmel_spec.max()))'  
  
plt.imshow(logmel_spec.numpy())  
plt.ylabel('Mels')  
plt.xlabel('Timesteps')  
  
plt.show()  
  
Spectrogram shape: torch.Size([64, 37])  
Values range: ((tensor(-8.3412), tensor(7.5945)))
```



Подготовка датасета

```
class AudioMNISTDataset(torch.utils.data.Dataset):
    def __init__(self, root, sr=16000):
        self.root = root
        self.sr = sr
        self.items = self.list_wavs_and_labels(root)

    def __len__(self):
        return len(self.items)

    def list_wavs_and_labels(self, root: str):
        base = Path(root)
        speakers = sorted([p for p in base.iterdir() if p.is_dir()])
        items = []
        for sp in speakers:
            for wav in sorted(sp.glob("**/*.wav")):
                # filename e.g., "9_10_0_0_1.wav" (digit_speaker_..)
                name = wav.stem.split("_")
                digit = int(name[0])
                speaker_id = sp.name
                items.append((str(wav), digit, speaker_id)) # [файл,
метка, айди спикера]
```

```

        return items

    # Загрузка и ресемплинг аудиофайла
    def load_wav(self, path):
        # wav, sr = torchaudio.load(path) # [C, T]
        y, sr = librosa.load(path, sr=None, mono=True)
        wav = torch.from_numpy(y).float().unsqueeze(0)

        if sr != self.sr:
            wav = torchaudio.functional.resample(wav, sr, self.sr)
        wav = wav.mean(dim=0, keepdim=True)
        return wav

    def __getitem__(self, idx):
        path, label, speaker = self.items[idx]
        wav = self.load_wav(path)

        return wav, label

    # Паддинг
    def collate(batch):
        wavs, labels = zip(*batch)
        wavs = nn.utils.rnn.pad_sequence([w.squeeze(0) for w in wavs],
            batch_first=True)
        labels = torch.tensor(labels, dtype=torch.long)
        return wavs, labels

    # Ensure data loading correctly
    _test_dataset = AudioMNISTDataset(root)
    _test_dataset[0]

    (tensor([[0.0003, 0.0005, 0.0004, ..., 0.0005, 0.0005, 0.0006]]), 0)

```

Создание датасета и разделение по говорящим

```

full_ds = AudioMNISTDataset(root=root)

all_speakers = sorted({spk for (_, _, spk) in full_ds.items})

NUM_TEST_SPEAKERS = 12
valid_speakers = set(all_speakers[-NUM_TEST_SPEAKERS:])

def split_indices_by_speaker(dataset: AudioMNISTDataset,
    test_speakers: set):
    train_idxes = []
    test_idxes = []
    for idx, (_, _, spk) in enumerate(dataset.items):
        if spk in test_speakers:
            test_idxes.append(idx)
        else:

```

```

        train_idxes.append(idx)
    return train_idxes, test_idxes

train_idxes, valid_idxes = split_indices_by_speaker(full_ds,
valid_speakers)
train_ds = Subset(full_ds, train_idxes)
valid_ds = Subset(full_ds, valid_idxes)

```

Сверточные энкодеры (ResNet1D, ResNet2D)

1. ResNet1D – for raw waveforms
2. ResNet2D – for spectrograms

```

class block(nn.Module):
    def __init__(self, in_channels, out_channels,
identity_downsample=None, stride=1):
        super(block, self).__init__()

        self.conv1 = nn.Conv1d(in_channels, out_channels,
kernel_size=1, stride=1, padding=0)
        self.bn1 = nn.BatchNorm1d(out_channels)

        self.conv2 = nn.Conv1d(out_channels, out_channels,
kernel_size=3, stride=stride, padding=1)
        self.bn2 = nn.BatchNorm1d(out_channels)

        self.identity_downsample = identity_downsample
        self.relu = nn.ReLU()

    def forward(self, x):
        identity = x

        x = self.conv1(x)
        x = self.bn1(x)
        x = self.relu(x)

        x = self.conv2(x)
        x = self.bn2(x)

        if self.identity_downsample is not None:
            identity = self.identity_downsample(identity)

        x += identity
        x = self.relu(x)

        return x

# Для аудиосигналов
class ResNet1D(nn.Module):
    # Resnet 18 [2, 2, 2, 2]

```

```

def __init__(self, block):
    super(ResNet1D, self).__init__()
    # for resnet18
    layers = [2, 2, 2, 2]
    self.expansion = 1

    self.in_channels = 64
    self.conv1 = nn.Conv1d(1, self.in_channels, kernel_size=7,
stride=2, padding=3, bias=False)
    self.bn1 = nn.BatchNorm1d(self.in_channels)
    self.relu = nn.ReLU()
    self.maxpool = nn.MaxPool1d(kernel_size=3, stride=2,
padding=1)

    self.layer1 = self._make_layer(block, layers[0], 64, stride=1)
    self.layer2 = self._make_layer(block, layers[1], 128,
stride=2)
    self.layer3 = self._make_layer(block, layers[2], 256,
stride=2)
    self.layer4 = self._make_layer(block, layers[3], 512,
stride=2)

    self.avgpool = nn.AdaptiveAvgPool1d(output_size=1)

    # size after avgpool = [32, 512, 1]

def forward(self, x):
    x = self.conv1(x)
    x = self.bn1(x)
    x = self.relu(x)
    x = self.maxpool(x)

    x = self.layer1(x)
    x = self.layer2(x)
    x = self.layer3(x)
    x = self.layer4(x)

    x = self.avgpool(x)

    return x

def _make_layer(self, block, num_residual_block, out_channels,
stride):
    identity_downsample = None
    layers = []

    if stride != 1:

```

```

        identity_downsample =
nn.Sequential(nn.Conv1d(self.in_channels,
out_channels*self.expansion,
kernel_size=1,
stride=stride,
bias=False),
nn.BatchNorm1d(out_channels*self.expansion),
)
        layers.append(
            block(self.in_channels, out_channels, identity_downsample,
stride)
        )
        self.in_channels = out_channels * self.expansion

        for i in range(1, num_residual_block):
            layers.append(block(self.in_channels, out_channels ))

        return nn.Sequential(*layers)

def CreateResNet1D():
    return ResNet1D(block)

# Блок для 2D ResNet
class block_resnet2d(nn.Module):
    def __init__(self, in_channels, out_channels,
identity_downsample=None, stride=1):
        super(block_resnet2d, self).__init__()

        self.conv1 = nn.Conv2d(in_channels, out_channels,
kernel_size=1, stride=1, padding=0)
        self.bn1 = nn.BatchNorm2d(out_channels)

        self.conv2 = nn.Conv2d(out_channels, out_channels,
kernel_size=3, stride=stride, padding=1)
        self.bn2 = nn.BatchNorm2d(out_channels)

        self.identity_downsample = identity_downsample
        self.relu = nn.ReLU()

    def forward(self, x):
        identity = x

        x = self.conv1(x)
        x = self.bn1(x)
        x = self.relu(x)

```

```

        x = self.conv2(x)
        x = self.bn2(x)

        if self.identity_downsample is not None:
            identity = self.identity_downsample(identity)

        x += identity
        x = self.relu(x)

        return x

# Для спектрограмм
class ResNet(nn.Module):
    # Resnet 18 [2, 2, 2, 2]
    def __init__(self, block, image_channels):
        super(ResNet, self).__init__()
        # for resnet18
        layers = [2, 2, 2, 2]
        self.expansion = 1

        self.in_channels = 64
        self.conv1 = nn.Conv2d(image_channels, self.in_channels,
kernel_size=7, stride=2, padding=3, bias=False)
        self.bn1 = nn.BatchNorm2d(self.in_channels)
        self.relu = nn.ReLU()
        self.maxpool = nn.MaxPool2d(kernel_size=3, stride=2,
padding=1)

        self.layer1 = self._make_layer(block, layers[0], 64, stride=1)
        self.layer2 = self._make_layer(block, layers[1], 128,
stride=2)
        self.layer3 = self._make_layer(block, layers[2], 256,
stride=2)
        self.layer4 = self._make_layer(block, layers[3], 512,
stride=2)

        self.avgpool = nn.AdaptiveAvgPool2d((1,1))

    def forward(self, x):
        x = self.conv1(x)
        x = self.bn1(x)
        x = self.relu(x)
        x = self.maxpool(x)

        x = self.layer1(x)
        x = self.layer2(x)
        x = self.layer3(x)

```

```

        x = self.layer4(x)

        x = self.avgpool(x)

        return x

    def _make_layer(self, block, num_residual_block, out_channels,
stride):
        identity_downsample = None
        layers = []

        if stride != 1:
            identity_downsample =
nn.Sequential(nn.Conv2d(self.in_channels,
out_channels*self.expansion,
kernel_size=1,
stride=stride,
bias=False),
nn.BatchNorm2d(out_channels*self.expansion),
)
        layers.append(
            block(self.in_channels,out_channels, identity_downsample,
stride)
        )
        self.in_channels = out_channels * self.expansion

        for i in range(1, num_residual_block):
            layers.append(block(self.in_channels,out_channels ))

        return nn.Sequential(*layers)

def CreateResNet2D(img_channels=3):
    return ResNet(block_resnet2d, image_channels=img_channels)

```

Реализация модели

```

class Net(nn.Module):

    def __init__(self, img_channels = 1, num_classes = 35):
        super(Net, self).__init__()
        ##### ENCODER
        #####
        self.resnet_1D = CreateResNet1D()
        self.resnet_2D = CreateResNet2D(img_channels=img_channels)

```

```

        self.output = nn.Sequential(
            nn.Linear(512, 256),
            nn.BatchNorm1d(256),
            nn.ReLU(),
            nn.Linear(256, 256),
            nn.BatchNorm1d(256),
            nn.ReLU(),
            nn.Linear(256, 128)
        )

#####

    def forward(self, input_spectrogram, input_audio):

        audio = self.resnet_1D(input_audio)
        audio = audio.squeeze()
        spectrograms = self.resnet_2D(input_spectrogram)
        spectrograms = spectrograms.squeeze()

        audio_emb = self.output(audio)
        specs_emb = self.output(spectrograms)

        return audio_emb, specs_emb, audio, spectrograms

# Оценки качества эмбеддингов
class EvaluationHead(nn.Module):

    def __init__(self, num_classes = 35):
        super(EvaluationHead, self).__init__()

        self.evaluation = nn.Sequential(
            nn.Linear(512, num_classes)
        )

    def forward(self, x):
        x = self.evaluation(x)
        return x

# Supervise модель
class SupervisedResNet1D(nn.Module):
    def __init__(self, num_classes=10):
        super(SupervisedResNet1D, self).__init__()

        self.encoder = CreateResNet1D()
        self.classifier = nn.Linear(512, num_classes)

    def forward(self, x):
        if x.dim() == 2:

```

```

        x = x.unsqueeze(1) # [batch, 1, time]
        features = self.encoder(x)
        features = features.squeeze() # [batch, 512]
        output = self.classifier(features)
        return output

# Отрисовка графиков

from IPython.display import clear_output

def get_plot(train_losses, val_accs):
    fig, (ax1, ax2) = plt.subplots(nrows=1, ncols=2, figsize=(15, 6))

    ax1.plot(train_losses, color='g', marker='o', markersize=4)
    ax1.set_title('Losses')
    ax1.set_xlabel('epoch')
    ax1.set_ylabel('loss')
    ax1.grid(True)

    ax2.plot(val_accs, color='r', marker='o', markersize=4)
    ax2.set_title('Accuracy')
    ax2.set_xlabel('epoch')
    ax2.set_ylabel('accuracy')
    ax2.grid(True)

    plt.tight_layout()

    clear_output(wait=True)

    plt.show()

# Сохранение модели

def save_model(model, optimizer, epoch, loss, accuracy, model_name,
save_dir='models/', is_best=False, is_final=False):

    checkpoint = {
        'epoch': epoch,
        'model_state_dict': model.state_dict(),
        'optimizer_state_dict': optimizer.state_dict(),
        'loss': loss,
    }

    if accuracy is not None:
        checkpoint['accuracy'] = accuracy

    # Сохраняем модель каждые 5 эпох
    if (epoch + 1) % 5 == 0:
        checkpoint_path =
f'{save_dir}/{model_name}_epoch_{epoch+1}.pt'

```

```

    torch.save(checkpoint, checkpoint_path)

    # Сохраняем лучшую модель
    if is_best:
        best_path = f'{save_dir}/{model_name}_best.pt'
        torch.save(checkpoint, best_path)

    # Сохраняем финальную модель
    if is_final:
        final_path = f'{save_dir}/{model_name}_final.pt'
        torch.save(checkpoint, final_path)

    return checkpoint

# Обучение для Supervised

def train_supervised(model, train_loader, val_loader, n_epoch=20,
                    lr=0.001): # n_epoch=20
    model = model.to(device)
    criterion = nn.CrossEntropyLoss(label_smoothing=0.1) #
    label_smoothing=0.1
    optimizer = optim.Adam(model.parameters(), lr=lr)

    train_losses = []
    val_accs = []
    best_accuracy = 0

    for epoch in range(n_epoch):
        model.train()
        running_loss = 0.0
        progress_bar = tqdm(train_loader, desc=f'Epoch
{epoch+1}/{n_epoch}',
                            # unit='batch', leave=False
# тест
                            )

        for audio, labels in progress_bar:
            audio, labels = audio.to(device), labels.to(device)

            if audio.dim() == 2:
                audio = audio.unsqueeze(1)

            optimizer.zero_grad()
            outputs = model(audio)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()

            running_loss += loss.item()

```

```

avg_loss = running_loss / len(train_loader)
train_losses.append(avg_loss)

# Валидация
model.eval()
correct = 0
total = 0

val_progress = tqdm(val_loader, desc=f'Eval
{epoch+1}/{n_epoch}')

with torch.no_grad():
    for audio, labels in val_progress:
        audio, labels = audio.to(device), labels.to(device)
        outputs = model(audio)
        _, predicted = torch.max(outputs.data, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

accuracy = correct / total
val_accs.append(accuracy)

# Сохранение модели

# Проверка is_best
is_best = accuracy > best_accuracy
if is_best:
    best_accuracy = accuracy

# Проверка is_final
is_final = (epoch == n_epoch - 1)

save_model(
    model=model,
    optimizer=optimizer,
    epoch=epoch,
    loss=avg_loss,
    accuracy=accuracy,
    model_name='supervised',
    save_dir='models/',
    is_best=is_best,
    is_final=is_final
)

get_plot(train_losses, val_accs)

print(f'Epoch {epoch+1}: Loss = {avg_loss:.4f}, Accuracy =
{accuracy:.4f}')

```

```

    return train_losses, val_accs

# Инициализация загрузчиков для Supervised

batch_size = 64

train_loader = DataLoader(train_ds, batch_size=batch_size,
                           shuffle=True, collate_fn=collate)
val_loader = DataLoader(valid_ds, batch_size=batch_size,
                        shuffle=False, collate_fn=collate)

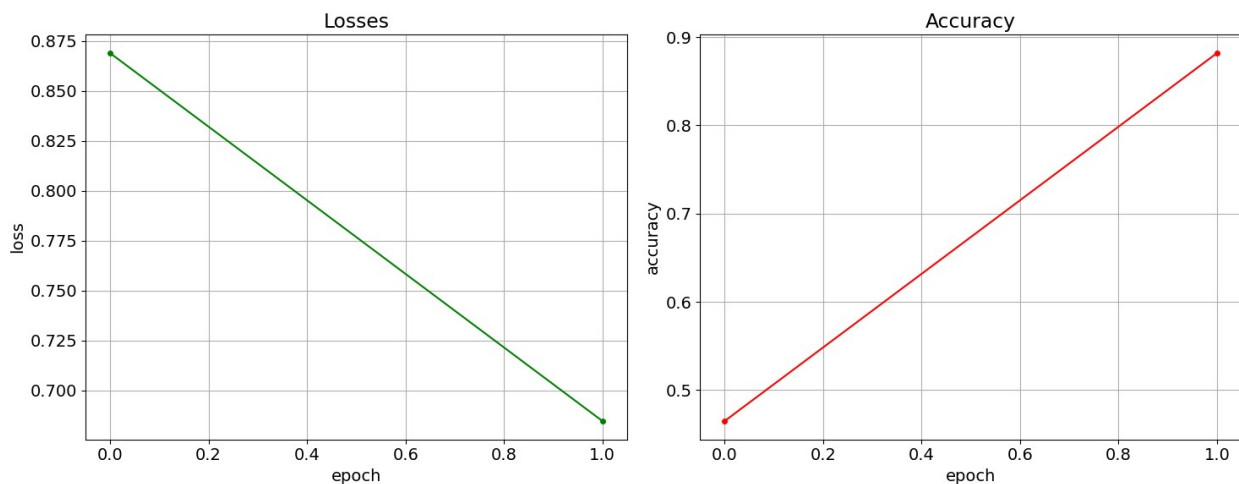
# stop

# Инициализация и обучение Supervised

torch.cuda.empty_cache()

supervised_model = SupervisedResNet1D(num_classes=10)
train_losses_sup, val_accs_sup = train_supervised(supervised_model,
                                                  train_loader, val_loader, n_epoch=2) # n_epoch=20 15

```



Epoch 2: Loss = 0.6846, Accuracy = 0.8818

```

# Метрики Supervised

print('\nМетрики на последней эпохе')
print(f'Loss: {train_losses_sup[-1]}')
print(f'Accuracy: {val_accs_sup[-1]}')

print('\nЛучшие метрики')
print(f'Loss: {min(train_losses_sup)}')
print(f'Accuracy: {max(val_accs_sup)}')

```

```
Loss: 0.684618317604065
Accuracy: 0.8818333333333334
```

```
# stop
```

Task 2 Contrastive Learning (InfoNCE) (2 points)

Train the Multi-Format Contrastive Learning of Audio Representations model (implemented during the seminar) until convergence. Plot the loss and accuracy curves to verify that the training process has stabilized.

Модель учится различать похожие и разные пары. Используется InfoNCE loss с температурным масштабированием.

Лосс для контрастивного обучения

```
# Лосс
def device_as(t1, t2):
    return t1.to(t2.device)

class ContrastiveLoss(nn.Module):
    def __init__(self, temperature=0.5):
        super().__init__()
        self.temperature = temperature # масштабирования схожести

    # Косинусная схожесть между векторами
    def calc_similarity_batch(self, a, b):
        rep = torch.cat([a, b])
        return F.cosine_similarity(rep.unsqueeze(1), rep.unsqueeze(0),
dim=2)

    def forward(self, proj_1, proj_2):
        batch_size = proj_1.shape[0]
        z_i = F.normalize(proj_1, p=2, dim=1)
        z_j = F.normalize(proj_2, p=2, dim=1)

        similarity_matrix = self.calc_similarity_batch(z_i, z_j)

        sim_ij = torch.diag(similarity_matrix, batch_size)
        sim_ji = torch.diag(similarity_matrix, -batch_size)

        positives = torch.cat([sim_ij, sim_ji])
        nominator = torch.exp(positives / self.temperature)

        mask = (~torch.eye(batch_size*2, batch_size*2).bool()).float()
        mask = device_as(mask, similarity_matrix)

        denominator = mask * torch.exp(similarity_matrix /
self.temperature)
        all_losses = -torch.log(nominator / torch.sum(denominator,
```

```

dim=1))
    loss = torch.sum(all_losses) / (2 * batch_size)

    return loss

```

Аугментации

Функции аугментации

```

def fade_in_out(audio):
    """
    Fade in/out (FD): gradually increases/decreases the intensity of
    the audio in the\n
    beginning/end of the audio signal.\n
    The degree of the fade was either linear, logarithmic or
    exponential (applied\n
    with uniform probability of 1/3). The size of the fade for either
    side of the\n
    audio signal could at maximum reach half of the audio signal. The
    size of the\n
    fade was another random parameter picked for each sample.
    """
    assert audio != None, "audio should not be None"

    _fade_shape = ['linear', 'logarithmic', 'exponential']
    _fade_size = [i for i in range(1, int(audio.shape[2]/2))]
    transform = T.Fade(fade_in_len=random.choice(_fade_size),
                      fade_out_len=random.choice(_fade_size),
                      fade_shape=random.choice(_fade_shape))
    transform = transform.to(audio.device)
    waveform_fade_in_out = transform(audio)

    return waveform_fade_in_out

def timemasking(waveform, batch_size, sample_rate=16000):
    """
    Time masking: given an audio signal, in this transformation we
    randomly select a small\n
    segment of the full signal and set the signal values in that
    segment to normal noise or a\n
    constant value. In our implementation, we not only randomly
    selected the location of the\n
    masked segment but also we randomly selected the size of the
    segment. The size of the \n
    masked segment was set to maximally be 1/8 of the input signal.
    """
    """max_mask = int(sample_rate/8)*torch.ones(size=[batch_size])
    pos_iniziale = torch.randint(low=0, high=sample_rate,
    size=[batch_size])

```

```

    min_mask = sample_rate-pos_iniziale
    min_elements = torch.min(min_mask,max_mask)
    pos_finale = pos_iniziale+min_elements.to(torch.int)
    indices =
torch.arange(sample_rate).unsqueeze(0).expand(batch_size, -1)
    range_mask = (indices >= pos_iniziale.unsqueeze(1)) & (indices <=
pos_finale.unsqueeze(1))
    range_mask = range_mask[:,None,:]
    signal_[range_mask] = 0

    return signal_"""

bs, ch, length = waveform.shape
mask_len = length // 8
augmented = waveform.clone()
for i in range(bs):
    start = random.randint(0, length - mask_len)
    augmented[i, :, start:start + mask_len] = 0.0

    return augmented

# Добавим еще аугментаций

def pitchshift(audio, SAMPLE_RATE=16000):
    """
    Pitch Shift (PS): randomly raises or lowers the pitch of the audio
    signal.\n
    Based on experimental observation,we found the range of pitch
    shifts that main-tained\n
    the overall coherency of the input audio was in the range [-15,
    15] semitones.

    Attributes:
    - :param audio: audio tensor
    - :param SAMPLE_RATE: Sample rate, default=16000
    - :param shift: Pitch shift
    - :return: describe what it returns
    """
    assert audio != None, "audio should not be None"
    # Делаем случайный сдвиг
    shift = random.randint(-15, 15)
    transform = transforms.PitchShift(sample_rate=SAMPLE_RATE,
n_steps=shift)
    transform = transform.to(audio.device)
    waveform_shift = transform(audio)

    return waveform_shift

def add_white_noise(signal):

```

```

    """
    Noise Injection: mix the audio signal with random white, brown and
    pink noise.\n
    In our implementation, the intensity of the noise signal was
    randomly selected based\n
    on the strength of signal-to-noise ratio. Applied white, brown, or
    pink depending\n
    on an additional random parameter sampled from uniform
    distribution (Mixed Noise).
    """
    # Случайный уровень шума
    noise_level = random.uniform(0.01, 0.1)

    noise = torch.randn_like(signal)*torch.std(signal) * noise_level
    noisy_signal = signal + noise

    return noisy_signal

```

Создаем спектрограммы и аугментированные аудио

```

def createModelInput(audio, mel_transform, stft_transform=None,
augmentation=True):

    audio = audio.unsqueeze(1) # [batch, 1, time]

    # Применяем аугментации если нужно
    if augmentation:
        # Создаем случайный набор аугментаций
        if random.random() < 0.9:
            audio = fade_in_out(audio)

        if random.random() < 0.9:
            audio = timemasking(audio, audio.shape[0])

        if random.random() < 0.3:
            audio = pitchshift(audio)

        if random.random() < 0.3:
            audio = add_white_noise(audio)

    # Создаем спектрограммы
    spectrograms = mel_transform(audio.squeeze(1))
    spectrograms = spectrograms.unsqueeze(1)
    spectrograms = spectrograms.to(audio.device)

    return spectrograms, audio

```

Функция create создает модель, loss, оптимизатор и mel-трансформацию

```

# Создает модель Contrastive, loss, оптимизатор и mel-трансформацию
def create(config):
    model = Net(img_channels=config.IMG_CHANNEL,
num_classes=config.CLASSES).to(device)
    loss = ContrastiveLoss()
    mel_transform = LogMelSpectrogram(sample_rate=16000, n_fft=2048,
hop_length=128, n_mels=128, f_min=40, f_max=8000,
mel_scale="slaney").to(device)
    optimizer = optim.Adam(model.parameters(), lr=config.LR,
betas=(config.B1, config.B2), weight_decay=config.WEIGHT_DECAY)
    return model, loss, optimizer, mel_transform

# Оценка качества эмбедингов
def evaluationphase(model, val_loader, config, mel_transform,
stft_transform=None):

    # # Очистка кэша # тест
    # torch.cuda.empty_cache() # тест

    model.eval()
    for param in model.parameters():
        param.requires_grad = False

    # Обучение линейной головы на замороженных эмбедингах
    def train_head(model, trainloader):
        modelEvaluation =
EvaluationHead(num_classes=config.CLASSES).to(device)
        optimizer = optim.Adam(modelEvaluation.parameters(),
lr=config.LR)
        criterion = torch.nn.CrossEntropyLoss(label_smoothing=0.1) #
label_smoothing=0.1

        for epoch in range(config.EVAL_EPOCHS):
            losses = []
            progress_bar = tqdm(trainloader, desc=f'Head Epoch
{epoch+1}/{config.EVAL_EPOCHS}',
unit='batch', leave=False)

            for audio, labels in progress_bar:
                optimizer.zero_grad()
                audio = audio.to(device)
                spectrograms, audios = createModelInput(audio,
mel_transform, augmentation=False)
                labels_cat = torch.cat([labels, labels],
dim=0).to(device)
                with torch.no_grad():
                    _, _, frozen_audio, frozen_spects =
model(spectrograms, audios)
                inputs = torch.cat([frozen_audio, frozen_spects],

```

```

dim=0)

        outputs = modelEvaluation(inputs)
        loss = criterion(outputs, labels_cat)
        loss.backward()
        optimizer.step()
        losses.append(loss.item())

        progress_bar.set_postfix({
            'loss': f'{loss.item():.4f}',
            'avg_loss': f'{np.mean(losses):.4f}'
        })

    progress_bar.close()

    ## Очистка кэша                                # тест
    # torch.cuda.empty_cache()                        # тест

    return modelEvaluation

# Оценка точности классификатора
def evaluation(model, model_eval, dataloader):
    model_eval.eval()

    progress_bar = tqdm(dataloader, desc='Evaluating',
unit='batch', leave=False)

    total = 0
    correct = 0
    with torch.no_grad():
        for audio, labels in progress_bar:
            audio = audio.to(device)
            spectrograms, audios = createModelInput(audio,
mel_transform, augmentation=False)
            labels_cat = torch.cat([labels, labels],
dim=0).to(device)
            _, _, frozen_audio, frozen_spects = model(spectrograms,
audios)
            inputs = torch.cat([frozen_audio, frozen_spects],
dim=0)

            outputs = model_eval(inputs)
            _, predicated = torch.max(outputs.data, 1)
            total += labels_cat.size(0)
            correct += (predicated == labels_cat).sum().item()

            current_acc = correct / total if total > 0 else 0
            progress_bar.set_postfix({'accuracy':
f'{current_acc:.4f}'})

```

```

        progress_bar.close()

        # # Очистка кэша                                # тест
        # torch.cuda.empty_cache()                        # тест

        return correct/total

    model_ = train_head(model, val_loader)
    accuracy_test = evaluation(model, model_, val_loader)
    print(f"Accuracy on validation: {accuracy_test}")
    model.train()
    for param in model.parameters():
        param.requires_grad = True

    # # Очистка кэша                                # тест
    # torch.cuda.empty_cache()                        # тест

    return accuracy_test

# Обучение модели contrastive

def train_contrastive(model, class, optimizer, trainloader, valloader,
config, mel_transform, stft_transform):
    scaler = torch.amp.GradScaler()
    train_losses = []
    val_accs = []
    best_accuracy = 0

    for epoch in range(config.EPOCHS):
        model.train()
        losses = []
        progress_bar = tqdm(trainloader, desc=f'Epoch
{epoch+1}/{config.EPOCHS}')

        for audio, _ in progress_bar:
            audio = audio.to(device)
            optimizer.zero_grad()

            spectrograms, audios = createModelInput(audio,
mel_transform, augmentation=True)

            with torch.amp.autocast(device_type=str(device)):
                audio_emb, spect_emb, _, _ = model(spectrograms,
audios)

                loss = closs(audio_emb, spect_emb)

            scaler.scale(loss).backward()
            scaler.step(optimizer)
            scaler.update()

```

```

        losses.append(loss.item())

        # progress_bar.set_postfix(loss=loss.item())

        progress_bar.set_postfix({
            'loss': f'{loss.item():.4f}',
            'avg_loss': f'{np.mean(losses):.4f}'
        })

    progress_bar.close()

    avg_loss = np.mean(losses)
    train_losses.append(avg_loss)

    # # Очистка кэша                                # тест
    # torch.cuda.empty_cache()                        # тест

    # Evaluation каждые 5 эпох
    if epoch % 5 == 0 or epoch == config.EPOCHS - 1:
        torch.cuda.empty_cache()
        accuracy_test = evaluationphase(model, valloader, config,
mel_transform, stft_transform)
        val_accs.append(accuracy_test)

    # Сохранение модели

    # Проверка is_best
    is_best = accuracy_test > best_accuracy
    if is_best:
        best_accuracy = accuracy_test

    # Проверка is_final
    is_final = (epoch == config.EPOCHS - 1)

    save_model(
        model=model,
        optimizer=optimizer,
        epoch=epoch,
        loss=avg_loss,
        accuracy=accuracy_test,
        model_name='contrastive',
        save_dir='models/',
        is_best=is_best,
        is_final=is_final
    )

    if accuracy_test > best_accuracy:
        best_accuracy = accuracy_test
        torch.save(model.state_dict(),
f"models/best_{config.MODEL_TITLE}.pt")

```

```

        print(f'Epoch {epoch+1}: Loss = {avg_loss:.4f}, Accuracy =
{accuracy_test:.4f}')
    else:
        val_accs.append(val_accs[-1] if val_accs else 0)
        print(f'Epoch {epoch+1}: Loss = {avg_loss:.4f}')

    # # Очистка кэша                                # тест
    # torch.cuda.empty_cache()                        # тест

    get_plot(train_losses, val_accs)
    return train_losses, val_accs

# Конфигурация для Contrastive

hyperparameters = {
    'LR': 3e-4,
    'WEIGHT_DECAY': 1e-6,
    'B1': 0.9,
    'B2': 0.999,
    'EPOCHS': 2, # 20 15
    'BATCH_SIZE': 256,
    'IMG_CHANNEL': 1,
    'CLASSES': 10,
    'EVAL_BATCH': 512,
    'EVAL_EPOCHS': 5, # 5 1
    'MODEL_TITLE': 'contrastive'
}

contrastive_config = DictConfig(hyperparameters)

train_loader = DataLoader(train_ds,
batch_size=contrastive_config.BATCH_SIZE, shuffle=True,
collate_fn=collate)
val_loader = DataLoader(valid_ds,
batch_size=contrastive_config.EVAL_BATCH, shuffle=False,
collate_fn=collate)

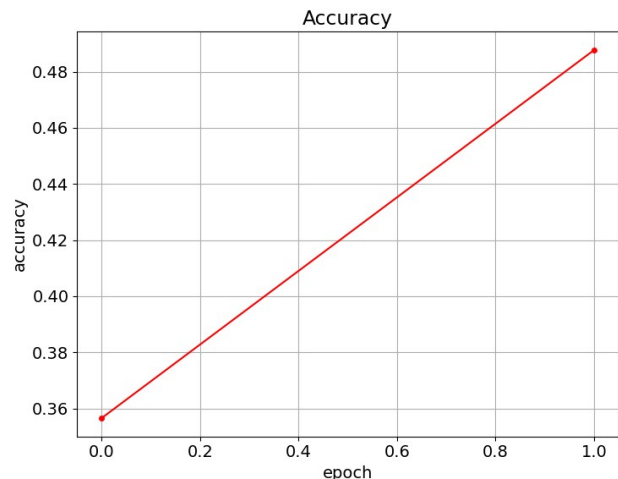
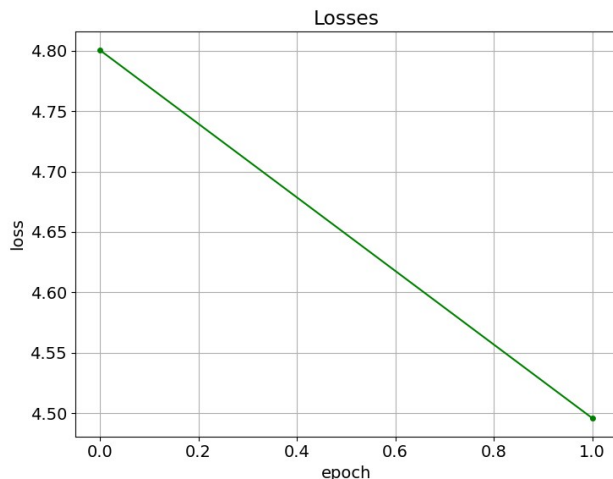
model, loss, optimizer, mel_transform = create(contrastive_config)

# Обучение Contrastive

torch.cuda.empty_cache()

train_losses_cont, val_accs_cont = train_contrastive(model, loss,
optimizer, train_loader, val_loader, contrastive_config,
mel_transform, None)

```



Метрики Contrastive

```
print('\nМетрики на последней эпохе')
print(f'Loss: {train_losses_cont[-1]}')
print(f'Accuracy: {val_accs_cont[-1]}')

print('\nЛучшие метрики')
print(f'Loss: {min(train_losses_cont)}')
print(f'Accuracy: {max(val_accs_cont)}')
```

Loss: 4.495972572488988
Accuracy: 0.4876666666666667

Task 3 Non-Contrastive Learning (NCL) (4 points)

Replace the InfoNCE loss in Multi-Format Contrastive Learning of Audio Representations with a Non-Contrastive Learning method. Train the model until convergence, then plot the loss and accuracy curves to check that training has stabilized.

You can try one of the following NCL methods:

- BYOL([paper](#))
- SimSiam([paper](#))
- Barlow Twins([paper](#))
- VicReg([paper](#))

Feel free to use a more recent Non-Contrastive approach if you prefer—just explain briefly why you chose it.

Из-за скорости обучения выберу SimSiam. Асимметричная архитектура, не требует отрицательных примеров.

```
class SimSiamLoss(nn.Module):
    def __init__(self, predictor_hidden_dim=256):
```

```

    super().__init__()
    self.predictor = nn.Sequential(
        nn.Linear(128, predictor_hidden_dim),
        nn.BatchNorm1d(predictor_hidden_dim),
        nn.ReLU(),
        nn.Linear(predictor_hidden_dim, 128)
    )

def forward(self, z1, z2):
    if next(self.predictor.parameters()).device != z1.device:
        self.predictor = self.predictor.to(z1.device)

    # Симметричная loss с stop-gradient
    loss = 0.5 * (self._calculate_loss(z1, z2.detach()) +
                  self._calculate_loss(z2, z1.detach()))
    return loss

    # Отрицательная косинусная схожесть между предсказанием и целью
def _calculate_loss(self, p, z):
    p = self.predictor(p)
    p = F.normalize(p, dim=1)
    z = F.normalize(z, dim=1)
    return - (p * z).sum(dim=1).mean()

# Два энкодерами (1D и 2D) и общая проекционная голова
# На выходе проекции для аудио и спектрограмм
class SimSiamModel(nn.Module):
    def __init__(self, img_channels=1):
        super().__init__()
        self.resnet_1D = CreateResNet1D()
        self.resnet_2D = CreateResNet2D(img_channels=img_channels)
        self.projection = nn.Sequential(
            nn.Linear(512, 256),
            nn.BatchNorm1d(256),
            nn.ReLU(),
            nn.Linear(256, 256),
            nn.BatchNorm1d(256),
            nn.ReLU(),
            nn.Linear(256, 128)
        )

    def forward(self, spectrogram, audio):
        audio_features = self.resnet_1D(audio).squeeze()
        spec_features = self.resnet_2D(spectrogram).squeeze()
        z_audio = self.projection(audio_features)
        z_spec = self.projection(spec_features)
        return z_audio, z_spec

```

```

# Создаем модель simsiam, loss, оптимизатор и mel-трансформацию

def create_simsiam(config):
    model = SimSiamModel(img_channels=config.IMG_CHANNEL).to(device)
    loss = SimSiamLoss(predictor_hidden_dim=256)
    mel_transform = LogMelSpectrogram(sample_rate=16000, n_fft=2048,
hop_length=128,
                                n_mels=128, f_min=40,
f_max=8000, mel_scale="slaney").to(device)
    optimizer = optim.Adam(model.parameters(), lr=config.LR,
betas=(config.B1, config.B2),
                                weight_decay=config.WEIGHT_DECAY)

    return model, loss, optimizer, mel_transform

```

При оценке качества эмбедингов используется только валидационная выборка, так как SimSiam обучается без меток

```

# Оценка качества эмбедингов SimSiam с помощью линейного
классификатора

def evaluate_simsiam(model, val_loader, config, mel_transform):

    # # Очистка кэша                                # тест
    # torch.cuda.empty_cache()                        # тест

    model.eval()

    val_features = []
    val_labels = []

    progress_bar = tqdm(val_loader, desc="Extracting features",
unit='batch', leave=False)

    with torch.no_grad():
        for audio, labels in progress_bar:
            audio = audio.to(device)
            spectrograms, audios = createModelInput(audio,
mel_transform, None, augmentation=False)
            audio_features = model.resnet_1D(audios).squeeze()
            spec_features = model.resnet_2D(spectrograms).squeeze()
            combined = torch.cat([audio_features, spec_features],
dim=1)
            val_features.append(combined.cpu())
            val_labels.append(labels)

        progress_bar.set_postfix({'samples': len(val_features)})

    progress_bar.close()

```

```

X_val = torch.cat(val_features, dim=0).numpy()
y_val = torch.cat(val_labels, dim=0).numpy()

from sklearn.linear_model import LogisticRegression
classifier = LogisticRegression(max_iter=1000, random_state=42,
C=1.0)
classifier.fit(X_val, y_val)
accuracy = classifier.score(X_val, y_val)

model.train()

# # Очистка кэша # тест
# torch.cuda.empty_cache() # тест

return accuracy

# Обучение модели SimSiam

def train_simsiam(model, simsiam_loss, optimizer, trainloader,
valloader, config, mel_transform):
    scaler = torch.amp.GradScaler()
    train_losses = []
    val_accs = []
    best_accuracy = 0

    for epoch in range(config.EPOCHS):
        losses = []
        progress_bar = tqdm(trainloader, desc=f'Epoch
{epoch+1}/{config.EPOCHS}')

        for audio, _ in trainloader:
            optimizer.zero_grad()
            audio = audio.to(device)

            # Создание двух аугментированных версий для каждого аудио
            spectrograms1, audios1 = createModelInput(audio,
mel_transform, augmentation=True)
            spectrograms2, audios2 = createModelInput(audio,
mel_transform, augmentation=True)

            with torch.amp.autocast(device_type=str(device)):
                z1_audio, z1_spec = model(spectrograms1, audios1)
                z2_audio, z2_spec = model(spectrograms2, audios2)
                loss_audio = simsiam_loss(z1_audio, z2_audio)
                loss_spec = simsiam_loss(z1_spec, z2_spec)
                loss = loss_audio + loss_spec

            scaler.scale(loss).backward()
            scaler.step(optimizer)

```

```

        scaler.update()

        progress_bar.set_description(f"Epoch
{epoch+1}/{config.EPOCHS}")
        progress_bar.set_postfix(loss=loss.item())
        progress_bar.update(1)
        losses.append(loss.item())

        progress_bar.set_postfix({
            'loss': f'{loss.item():.4f}',
            'avg_loss': f'{np.mean(losses):.4f}'
        })
        progress_bar.update(1)

    avg_loss = np.mean(losses)
    train_losses.append(avg_loss)
    progress_bar.close()

    if epoch % 5 == 0 or epoch == config.EPOCHS - 1:
        torch.cuda.empty_cache()
        torch.save(model.state_dict(),
f"models/model_{config.MODEL_TITLE}.pt")
        accuracy_test = evaluate_simsiam(model, valloader, config,
mel_transform)
        val_accs.append(accuracy_test)

        # Сохранение модели

        # Проверка is_best
        is_best = accuracy_test > best_accuracy
        if is_best:
            best_accuracy = accuracy_test

        # Проверка is_final

        is_final = (epoch == config.EPOCHS - 1)
        save_model(
            model=model,
            optimizer=optimizer,
            epoch=epoch,
            loss=avg_loss,
            accuracy=accuracy_test,
            model_name='simsiam',
            save_dir='models/',
            is_best=is_best,
            is_final=is_final
        )

        print(f'Epoch {epoch+1}: Loss = {avg_loss:.4f}, Accuracy =
{accuracy_test:.4f}')

```

```

        else:
            val_accs.append(val_accs[-1] if val_accs else 0)
            print(f'Epoch {epoch+1}: Loss = {avg_loss:.4f}')

    # Финальная отрисовка графика
    get_plot(train_losses, val_accs)

    return train_losses, val_accs

# Конфигурация для SimSiam
hyperparameters_simsiam = {
    'LR': 3e-4,
    'WEIGHT_DECAY': 1e-6,
    'B1': 0.9,
    'B2': 0.999,
    'EPOCHS': 2, # 20 15
    'BATCH_SIZE': 256,
    'IMG_CHANNEL': 1,
    'CLASSES': 10,
    'EVAL_BATCH': 512,
    'EVAL_EPOCHS': 5, # 5 1
    'MODEL_TITLE': 'simsiam'
}

simsiam_config = DictConfig(hyperparameters_simsiam)

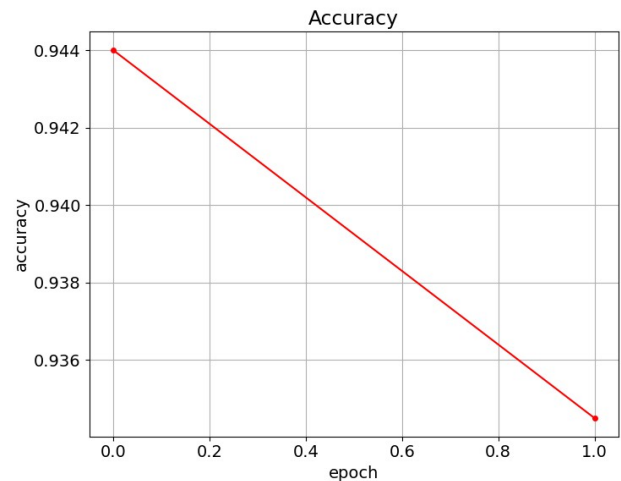
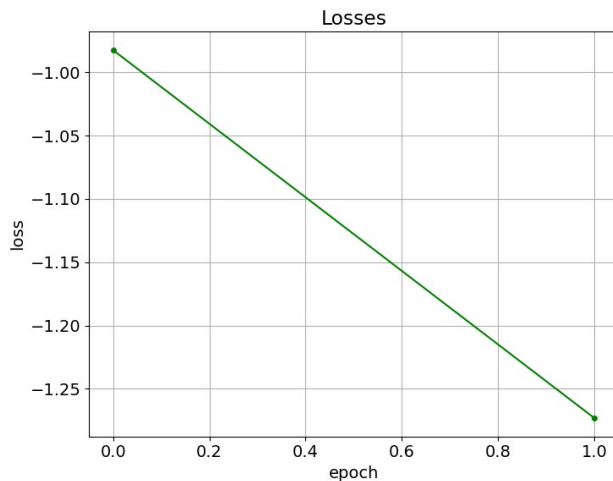
simsiam_model, simsiam_loss, simsiam_optimizer, simsiam_mel_transform
= create_simsiam(simsiam_config)

# Обучение SimSiam

torch.cuda.empty_cache()

train_losses_simsiam, val_accs_simsiam = train_simsiam(
    simsiam_model, simsiam_loss, simsiam_optimizer,
    train_loader, val_loader, simsiam_config, simsiam_mel_transform
)

```



```
# Метрики SimSiam
```

```
print('\nМетрики на последней эпохе')
print(f'Loss: {train_losses_simsiam[-1]}')
print(f'Accuracy: {val_accs_simsiam[-1]}')

print('\nЛучшие метрики')
print(f'Loss: {min(train_losses_simsiam)}')
print(f'Accuracy: {max(val_accs_simsiam)}')
```

```
Метрики на последней эпохе
Loss: -1.2730582293043746
Accuracy: 0.9345
```

```
Лучшие метрики
Loss: -1.2730582293043746
Accuracy: 0.944
```

Task 4 Comparison of training setups (2 points)

Evaluate and compare three training setups on the validation subset:

1. Supervised training
2. InfoNCE (contrastive learning)
3. Non-Contrastive Learning (NCL)

Use either the 2D or 1D encoder—or combine their embeddings. Identify which setup performs best and explain why it outperforms the others.

```
def extract_all_embeddings(model, dataloader, model_type,
mel_transform=None):
```

```

# # Очистка кэша                                # тест
# torch.cuda.empty_cache()                        # тест

model.eval()
embeddings_audio = []
labels_list = []

desc = f"Extracting {model_type} embeddings"
progress_bar = tqdm(dataloader, desc=desc, unit='batch',
leave=True)

with torch.no_grad():
    for audio, labels in progress_bar:
        audio = audio.to(device, non_blocking=True)

        if model_type == 'supervised':
            if audio.dim() == 2:
                audio = audio.unsqueeze(1)
                features = model.encoder(audio).squeeze()
                embeddings_audio.append(features.cpu())

            elif model_type == 'contrastive':
                spectrograms, audios = createModelInput(audio,
mel_transform, augmentation=False)
                audio_emb, spec_emb, audio_feat, spec_feat =
model(spectrograms, audios)
                combined = torch.cat([audio_feat, spec_feat], dim=1)
                embeddings_audio.append(combined.cpu())

            elif model_type == 'simsiam':
                spectrograms, audios = createModelInput(audio,
mel_transform, augmentation=False)
                audio_feat = model.resnet_1D(audios).squeeze()
                spec_feat = model.resnet_2D(spectrograms).squeeze()
                combined = torch.cat([audio_feat, spec_feat], dim=1)
                embeddings_audio.append(combined.cpu())

        labels_list.append(labels)

    progress_bar.set_postfix({
        'batch': len(embeddings_audio),
        'samples': len(embeddings_audio) *
dataloader.batch_size if embeddings_audio else 0
    })

progress_bar.close()

# # Очистка кэша                                # тест
# torch.cuda.empty_cache()                        # тест

```

```

        return torch.cat(embeddings_audio, dim=0).numpy(),
        torch.cat(labels_list, dim=0).numpy()

# Извлечение эмбедингов

supervised_embs, supervised_labels =
extract_all_embeddings(supervised_model, val_loader, 'supervised')
contrastive_embs, contrastive_labels = extract_all_embeddings(model,
val_loader, 'contrastive', mel_transform)
simsiam_embs, simsiam_labels = extract_all_embeddings(simsiam_model,
val_loader, 'simsiam', simsiam_mel_transform)

{"model_id": "ded67233e00a4b24aa786e30d488dad1", "version_major": 2, "version_minor": 0}

{"model_id": "40b6ae5222704ab4ad24f74aaf7d6bc4", "version_major": 2, "version_minor": 0}

{"model_id": "7ad0248d639241fe8d42812c129c9987", "version_major": 2, "version_minor": 0}

def plot_comparison_tsne(embeddings_dict):
    fig, axes = plt.subplots(1, 3, figsize=(18, 5))

    for idx, (name, (embs, labels)) in
enumerate(embeddings_dict.items()):
        tsne = TSNE(n_components=2, random_state=42, perplexity=30)
        embs_2d = tsne.fit_transform(embs)

        scatter = axes[idx].scatter(embs_2d[:, 0], embs_2d[:, 1],
c=labels, cmap='tab10', s=10,
alpha=0.7)
        axes[idx].set_title(f'{name}')
        axes[idx].set_xlabel('t-SNE 1')
        axes[idx].set_ylabel('t-SNE 2')
        plt.colorbar(scatter, ax=axes[idx])

    plt.tight_layout()
    plt.show();

embeddings_dict = {
    'Supervised': (supervised_embs, supervised_labels),
    'Contrastive (InfoNCE)': (contrastive_embs, contrastive_labels),
    'Non-Contrastive (SimSiam)': (simsiam_embs, simsiam_labels)
}

plot_comparison_tsne(embeddings_dict)

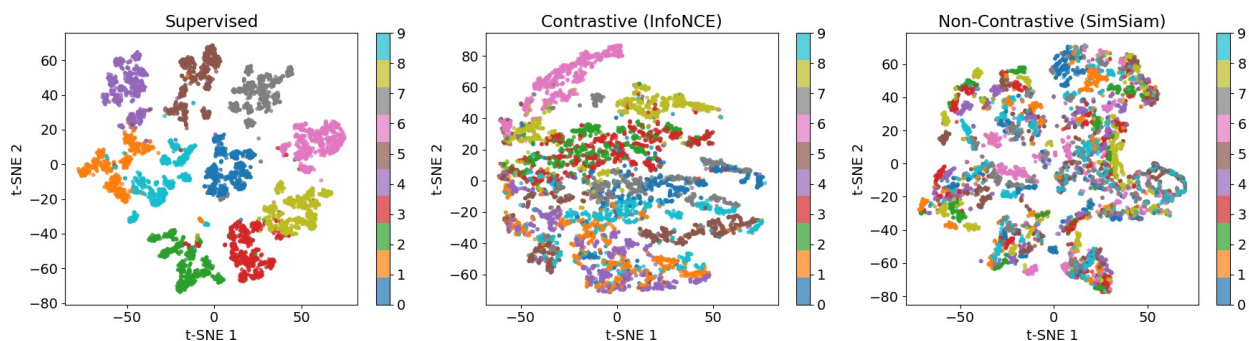
```

File "c:\Users\pong\anaconda3\envs\ml\Lib\site-packages\joblib\externals\loky\backend\context.py", line 257, in _count_physical_cores

```

cpu_info = subprocess.run(
    ~~~~~~
File "c:\Users\pong\anaconda3\envs\ml\Lib\subprocess.py", line 548,
in run
    with Popen(*popenargs, **kwargs) as process:
    ~~~~~~
File "c:\Users\pong\anaconda3\envs\ml\Lib\subprocess.py", line 1026,
in __init__
    self._execute_child(args, executable, preexec_fn, close_fds,
File "c:\Users\pong\anaconda3\envs\ml\Lib\subprocess.py", line 1538,
in _execute_child
    hp, ht, pid, tid = _winapi.CreateProcess(executable, args,
    ~~~~~~

```



```

# Оценка качества эмбедингов через k-NN классификацию
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

def evaluate_knn(train_embs, train_labels, test_embs, test_labels,
k=5):
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(train_embs, train_labels)
    predictions = knn.predict(test_embs)
    return accuracy_score(test_labels, predictions)

train_embs_sup, train_labels_sup =
extract_all_embeddings(supervised_model, train_loader, 'supervised')
train_embs_cont, train_labels_cont = extract_all_embeddings(model,
train_loader, 'contrastive', mel_transform)
train_embs_simsiam, train_labels_simsiam =
extract_all_embeddings(simsiam_model, train_loader, 'simsiam',
simsiam_mel_transform)

{"model_id": "1b5d7001aa184d398d373582a2ff8ec3", "version_major": 2, "vers
ion_minor": 0}

{"model_id": "0fd82b3893c340d68cb5ead42ffa7f4b", "version_major": 2, "vers
ion_minor": 0}

```

```

{"model_id": "9051d16e907c45c5a32f3e4a58d95848", "version_major": 2, "version_minor": 0}

# Оценка k-NN

knn_sup = evaluate_knn(train_embs_sup, train_labels_sup,
supervised_embs, supervised_labels)
print(f'Supervised k-NN Accuracy: {knn_sup:.4f}')

knn_cont = evaluate_knn(train_embs_cont, train_labels_cont,
contrastive_embs, contrastive_labels)
print(f'Contrastive (InfoNCE) k-NN Accuracy: {knn_cont:.4f}')

knn_simsiam = evaluate_knn(train_embs_simsiam, train_labels_simsiam,
simsiam_embs, simsiam_labels)
print(f'Non-Contrastive (SimSiam) k-NN Accuracy: {knn_simsiam:.4f}')

Supervised k-NN Accuracy: 0.9368
Contrastive (InfoNCE) k-NN Accuracy: 0.7553
Non-Contrastive (SimSiam) k-NN Accuracy: 0.5620

# Линейная классификация

from sklearn.linear_model import LogisticRegression

def evaluate_linear(train_embs, train_labels, test_embs, test_labels):
    lr = LogisticRegression(max_iter=1000, random_state=42)
    lr.fit(train_embs, train_labels)
    predictions = lr.predict(test_embs)
    return accuracy_score(test_labels, predictions)

linear_sup = evaluate_linear(train_embs_sup, train_labels_sup,
supervised_embs, supervised_labels)
print(f'Supervised Linear Accuracy: {linear_sup:.4f}')

linear_cont = evaluate_linear(train_embs_cont, train_labels_cont,
contrastive_embs, contrastive_labels)
print(f'Contrastive (InfoNCE) Linear Accuracy: {linear_cont:.4f}')

linear_simsiam = evaluate_linear(train_embs_simsiam,
train_labels_simsiam, simsiam_embs, simsiam_labels)
print(f'Non-Contrastive (SimSiam) Linear Accuracy:
{linear_simsiam:.4f}')

Supervised Linear Accuracy: 0.9523
Contrastive (InfoNCE) Linear Accuracy: 0.9373
Non-Contrastive (SimSiam) Linear Accuracy: 0.8493

```

- Supervised Learning: Хорошо работает, но требует много размеченных данных
- Contrastive Learning (InfoNCE): Учится различать похожие и разные пары, но требует больших батчей и тщательной настройки температуры

- Non-Contrastive Learning (SimSiam): Избегает коллапса представлений без отрицательных примеров, более стабилен в обучении

Supervised метод показал лучшие результаты благодаря прямому обучению на целевые метки и отсутствию потери информации при проекции.

Из-за нехватки времени на обучение исправленных моделей, приложу результаты предыдущих экспериментов:

WhatsApp Image 2026-04-07 at 11.31.16.jpeg

WhatsApp Image 2026-04-07 at 11.31.49.jpeg

WhatsApp Image 2026-04-07 at 11.32.11.jpeg

WhatsApp Image 2026-04-07 at 11.32.28.jpeg

WhatsApp Image 2026-04-07 at 11.32.44.jpeg

WhatsApp Image 2026-04-07 at 11.33.03.jpeg

WhatsApp Image 2026-04-07 at 11.33.21.jpeg